

UCS503 - Software Engineering

SyncDrive

Signal-Aware Navigation and Speed Optimization System

Submitted by:

1024030177 Tarun Mahindra

1024030178 Parth Puri

1024031077 Adhiraj Kapur

Group: BE Third Year, CSE

Submitted to:

Dr. Paramveer Sidhu

Mid-Semester Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1	Introduction	5
1.1	Background and Context	5
1.1.1	Problem Statement	5
1.2	Project Objectives	6
2	Methodology and Design	7
2.1	System Architecture	7
2.1.1	High-Level Design	7
2.1.2	Technical Stack	7
2.2	Implementation Details	7
2.2.1	Module 1: Green Wave Optimizer	7
2.2.2	Module 2: Normal GPS Simulation	8
3	Results and Evaluation	9
3.1	Testing Methodology	9
3.2	Performance Metrics	9
3.3	Analysis	10
4	Conclusion	11
4.1	Summary of Work	11
4.2	Key Findings	11
4.3	Future Work and Recommendations	11
4.4	Final Remarks	12
	Bibliography	13

Introduction

1.1 Background and Context

Urban commuters in Indian cities spend a significant portion of their travel time waiting at traffic signals. In a city like Chandigarh, whose grid-sector layout concentrates traffic through a small number of major intersections, the stop-and-go pattern is both predictable and avoidable. Standard navigation applications -- Google Maps, Apple Maps, and their equivalents -- select routes based on distance and estimated travel time, but do not account for how a driver's chosen speed interacts with the timing of upcoming traffic signals.

The concept of a green wave -- coordinating signal timings along a corridor so that a vehicle travelling at a fixed speed passes every light on green -- is well established in traffic engineering. SyncDrive inverts this concept for the individual driver: rather than the city adjusting signals to match a speed, the driver adjusts their speed and departure time to match the existing signal pattern.

1.1.1 Problem Statement

Existing navigation systems are signal-blind: they select a road but not the speed or departure time that would let the driver traverse it without stopping. This causes three compounding inefficiencies:

- Avoidable red-light stops: a driver arriving 30 seconds after a red began waits up to 90 seconds for no mechanical reason.
- Stop-and-go fuel waste: repeated acceleration from rest consumes significantly more fuel than steady-speed driving.
- No departure-time guidance: apps tell a driver which road to take, not when to leave or how fast to drive on it.

1.2 Project Objectives

1. Objective 1 -- Green wave optimization: Given a source, destination, and desired arrival time, compute the departure time and steady driving speed that minimises total journey time across all signals on the route.
2. Objective 2 -- Quantified comparison: Simulate a Normal GPS journey on the same route at the same departure time but at 50 km/h, compute red-light wait time, and present the time saved by SyncDrive side by side.
3. Objective 3 -- Working browser prototype: Deliver a single-file web application (no install, no backend, no API key) covering 7 Chandigarh locations, 42 directed routes, with an interactive dark map, signal timeline, and turn-by-turn directions.

Methodology and Design

2.1 System Architecture

The prototype is a single HTML file with embedded CSS and JavaScript. There is no backend, no build step, and no external API dependency beyond map tile fetching.

2.1.1 High-Level Design

1. Data Layer -- Static JavaScript objects: PLACES (7 locations), SIGNALS (16 traffic signals), ROUTES (42 directed route objects).
2. Algorithm Layer -- Two functions: `optimise()` (green wave optimizer) and `simNG()` (Normal GPS simulator).
3. Map Layer -- Leaflet.js 1.9.4 rendering an interactive map with place labels, signal markers, route polylines, and a speed HUD.
4. UI Layer -- Three-pane Apple Maps-inspired layout: nav rail, directions panel, map.

2.1.2 Technical Stack

- Language: Vanilla JavaScript (ES6+), HTML5, CSS3.
- Map library: Leaflet.js 1.9.4 (CDN).
- Map tiles: Esri World Dark Gray Canvas (free, no API key).
- Font: Inter via Google Fonts CDN.
- Build tools: None -- single `index.html` file.
- Deployment: GitHub Pages (<https://parthpuri.github.io/SyncDrive/>).
- Version control: Git / GitHub (<https://github.com/ParthPuri/SyncDrive>).

2.2 Implementation Details

2.2.1 Module 1: Green Wave Optimizer

The optimizer searches over a grid of candidate (speed, departure offset) pairs and returns the combination that minimises total journey time = drive time + signal wait time.

```
Speeds : [25,28,30,32,35,38,40,42,45] km/h
Offsets: 0 to 1500 s, step 15 s

for each speed s:
  driveSec = (distKm / s) * 3600
  for each offset d:
    dep = arrivalTime - driveSec - d
    elapsed = 0
    for each signal:
      phase = (dep + timeToSignal + elapsed) % cycle
      if green: greens++
      else: wait = nextGreen(sig,phase)
```

```
        elapsed += wait; totalwait += wait
    score = (driveSec + totalwait) - 0.5 * greens
    if score < best: save result
```

The elapsed accumulation is the critical correctness requirement: each red stop shifts all downstream signal arrival times. Without this propagation the signal counts are wrong.

2.2.2 Module 2: Normal GPS Simulation

The Normal GPS simulator uses the same departure time as the SyncDrive result but drives at 50 km/h. It hits signals at uncontrolled phases and accumulates real wait times using identical elapsed propagation. Using the same departure time is essential for a fair comparison: any difference in total time is attributable solely to speed choice and signal timing, not to when the driver left.

Results and Evaluation

3.1 Testing Methodology

Testing was conducted computationally using the simulator built into the prototype. All 42 directed routes were evaluated at a 09:00 arrival target.

- Unit Testing: Each signal phase function was verified manually against hand-computed values for known (departure, cycle, offset) combinations.
- Integration Testing: The optimizer and simulator were run on all 42 routes and checked for correctness (GreenWave total should never exceed Normal GPS total for the same departure).
- Boundary Testing: Departure times near midnight were checked to confirm modular arithmetic does not produce negative phase values.
- Browser Testing: Application loaded and verified in Chrome and Safari with no console errors.

3.2 Performance Metrics

Route	km	Normal GPS	SyncDrive	Saved
Sec 17 -> Sec 22	1.4	3.8 min	2.6 min	1.2 min
Sec 22 -> Sec 35	2.2	6.0 min	4.5 min	1.5 min
Sec 17 -> PGI	3.2	5.8 min	5.0 min	0.8 min
Sec 43 -> Elante	2.0	4.1 min	4.0 min	0.1 min
Sec 17 -> Tribune	5.1	8.2 min	7.7 min	0.5 min
Sec 22 -> PGI	4.1	7.9 min	7.0 min	0.9 min
Sec 35 -> Sec 43	2.8	5.9 min	4.9 min	1.0 min

Table 3.1: Journey time comparison: Normal GPS vs SyncDrive (09:00 arrival target). Based on simulated signal timings.

Metric	Target	Achieved
Routes covering all 7 locations	42	42
Signals modelled	≥ 10	16
SyncDrive beats or ties Normal GPS	100%	100%
App loads without server	Yes	Yes
No API key required	Yes	Yes

Table 3.2: Prototype objective achievement summary.

3.3 Analysis

All three objectives from Chapter 1 are fully met. The optimizer produces a numeric departure time and speed for every route pair. The comparison panel displays the time breakdown side-by-side. The prototype

loads in any browser with no installation.

The prototype exceeds the original scope: 7 locations, 16 signals, and 42 directed routes, versus the single test corridor described in the proposal. The UI additionally renders a per-signal timeline, turn-by-turn directions, and a speed HUD on the map.

Key bugs resolved during development:

- Tile provider API keys: Carto and Stadia began requiring keys mid-project. Resolved by switching to Esri Dark Gray Canvas (free, no key).
- Wrong optimizer objective: First version maximised green count, ignoring total time. Fixed by minimising (drive + wait).
- Missing elapsed propagation: Early builds did not accumulate wait time, causing wrong downstream phases. Fixed by threading elapsed through the signal loop.
- Unfair comparison baseline: Normal GPS simulation initially used arrival time as departure reference. Fixed to use same departure as SyncDrive.

Conclusion

4.1 Summary of Work

SyncDrive demonstrates that a driver can arrive at a destination faster by driving at a lower, steady speed timed to a signal wave, compared to rushing at the city speed limit and accumulating red-light wait time. The prototype delivers a working single-file web application covering Chandigarh's key corridors, with an interactive dark map, 42 routes, 16 simulated signals, a green wave optimizer, a Normal GPS comparison, and an Apple Maps-inspired interface -- all without a backend server or API key.

4.2 Key Findings

- Finding 1 -- Slower speed, faster arrival: On short routes (1-2 km), driving at 30-35 km/h with zero stops consistently beats 50 km/h with 2-3 red-light stops by 0.5-1.5 minutes.
- Finding 2 -- Elapsed propagation is non-negotiable: The single most important correctness requirement is accumulating wait time through the signal loop. Omitting it produces systematically optimistic green counts.
- Finding 3 -- Fair comparison requires equal departure time: Using the same departure for both strategies isolates the effect of speed and signal timing, making time savings directly attributable to the green wave approach.

4.3 Future Work and Recommendations

1. Live position tracking: Use the browser Geolocation API to track driver position and continuously recalculate speed as the journey progresses.
2. Departure countdown: Add a "Leave in X minutes" live timer based on the current system clock.
3. Animated route playback: Move a vehicle marker along the polyline at the recommended speed.
4. Real signal data: Source cycle lengths from OpenStreetMap `highway=traffic_signals` and `cycle_time` tags.
5. Peak-hour profiles: Model different cycle lengths for morning (08:00-10:00) and evening (17:00-20:00) rush hours.
6. OSRM routing integration: Replace hardcoded polylines with road-following routes from the OSRM engine.
7. Emergency vehicle coordination: Adjust nearby drivers' speed when an emergency vehicle is active on the corridor.

4.4 Final Remarks

The prototype validates the central hypothesis of SyncDrive within a controlled, simulated environment. The engineering challenge was not the map rendering or the UI -- it was getting the signal phase simulation

mathematically correct and ensuring the comparison between strategies was fair. Both issues were identified, debugged, and fixed during the five weeks of development documented in the team journals. The foundation is solid for the next phase of work.

Bibliography

- [OSM26] OpenStreetMap Contributors. OpenStreetMap. 2026. url: <https://www.openstreetmap.org>
- [Lea26] Agafonkin, Vladimir. Leaflet.js -- a JavaScript library for interactive maps. 2026. url: <https://leafletjs.com>
- [Esr26] Esri. ArcGIS World Dark Gray Canvas Basemap. 2026. url: <https://www.esri.com>
- [Lit66] Little, John D. C. "The Synchronization of Traffic Signals by Mixed-Integer Linear Programming." Operations Research 14.4 (1966): 568-594.